

Ingegneria de Software – Parte II

Altri pattern di refactoring



UNIVERSITÀ
DEGLI STUDI
DI BRESCIA

Prof. Alessandro Saetti

Tipi di refactoring



- **Per sistemare metodi troppo lunghi**
- Per sistemare le responsabilità mal attribuite
- Per riorganizzare i dati
- Per semplificare statement condizionali
- Per rendere le chiamate ai metodi più semplici
- Per sistemare le gerarchie

3

Replace method with method object

Situazione

Un metodo lungo usa variabili locali in un modo tale che non è possibile usare Extract method

Soluzione

Sposta il metodo in un proprio oggetto rendendo attributi tutte le variabili locali del metodo

Motivazione

Si vorrebbe accorciare il metodo ma questo è impedito dalla presenza di variabili locali usate dappertutto nel metodo

4

Come operare il refactoring?

1. Crea una classe con il nome uguale al nome del metodo
2. Inserisci un attributo final per la classe di partenza e un attributo per ogni parametro e variabile locale al metodo;
3. Inserisci un costruttore con i parametri passati al metodo;
4. Dichiarare un metodo compute() nella nuova classe
5. Sposta il corpo del metodo originale dentro compute(), sistemando le invocazioni ai metodi nell'oggetto di partenza
6. Compila e testa;
7. Rimpiazza il metodo originale con un metodo che istanzia la nuova classe ed invoca il metodo compute();
8. Okay, adesso è facile usare il pattern Extract method

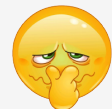
5

Esempio di Replace method with method object

Prima e per cominciare...

```
class Account { ...  
  int gamma (int inputVal, int quantity, int yearToDate) {  
    int importantValue1 = (inputVal * quantity) + delta();  
    int importantValue2 = (inputVal * yearToDate) + 100;  
    if ((yearToDate - importantValue1) > 100)  
      importantValue2 -= 20;  
    int importantValue3 = importantValue2 * 7;  
    // and so on.  
    return importantValue3 - 2 * importantValue1;  
  }  
}
```

Vorremmo applicare
Extract method ma l'uso
di variabili locali rende
l'estrazione difficile



```
class Gamma {  
  private final Account _account;  
  private int inputVal;  
  private int quantity;  
  private int yearToDate;  
  private int importantValue1;  
  private int importantValue2;  
  private int importantValue3;  
}
```

**Creo una nuova classe Gamma
Inserisco un attributo _account per la
classe Account e un attributo per le
variabili e i parametri di Gamma**

1,2

```
Gamma (Account source, int inputValArg, int quantityArg, int yearToDateArg) {  
  _account = source;  
  inputVal = inputValArg;  
  quantity = quantityArg;  
  yearToDate = yearToDateArg;  
}
```

**Definisco un costruttore
per Gamma**

3

...durante e dopo

```
class Gamma { ...  
  int compute () {  
    importantValue1 = (inputVal * quantity) + _account.delta();  
    importantValue2 = (inputVal * yearToDate) + 100;  
    if ((yearToDate - importantValue1) > 100)  
      importantValue2 -= 20;  
    int importantValue3 = importantValue2 * 7;  
    // and so on.  
    return importantValue3 - 2 * importantValue1;  
  }  
}
```

4,5

**Sposto il corpo del metodo
gamma dentro a compute
Sistemo l'invocazione al
metodo delta()**

6

```
class Account { ...  
  int gamma (int inputVal, int quantity, int yearToDate) {  
    return new Gamma(this, inputVal,  
      quantity, yearToDate).compute();  
  }  
}
```

**Istanzio Gamma e
delego a compute di
Gamma**

7

```
int compute () {  
  importantValue1 = (inputVal * quantity) + _account.delta();  
  importantValue2 = (inputVal * yearToDate) + 100;  
  importantThing();  
  int importantValue3 = importantValue2 * 7;  
  // and so on.  
  return importantValue3 - 2 * importantValue1;  
}  
  
void importantThing() {  
  if ((yearToDate - importantValue1) > 100)  
    importantValue2 -= 20;  
}
```

**Estraggo il metodo
importantThing**

8

Tipi di refactoring



- Per sistemare metodi troppo lunghi
- **Per sistemare le responsabilità mal attribuite**
- Per riorganizzare i dati
- Per semplificare statement condizionali
- Per rendere le chiamate ai metodi più semplici
- Per sistemare le gerarchie

Hide Delegate

Situazione

Un client sta chiamando un metodo di un oggetto (delegate) memorizzato in uno degli attributi della classe server

Soluzione

Crea dei metodi sulla classe server per nascondere il delegate

Motivazione

Se delegate cambia anche il client potrebbe dover cambiare
Il server dovrebbe arginare il cambiamento (**Indirection**)

Pattern correlati

Remove middle man (rimuove una classe che delega troppo)

9

Come operare il refactoring?

1. Per ciascun metodo dell'oggetto delegate, crea un metodo delegante nell'oggetto server
2. Sistema i client affinché chiamino i metodi deleganti
3. Compila e testa dopo ogni metodo delegante;
4. Se i client non hanno più bisogno del delegato, rimuovi dal server il metodo che lo espone
5. Compila e testa

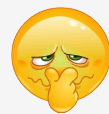
Esempio di Hide Delegate

Prima...

```
class Person { ...
    Department _department;
    public Department getDepartment() {
        return _department;
    }
    public void setDepartment(Department arg) {
        _department = arg;
    }
}
```

```
class Department { ...
    private String _chargeCode;
    private Person _manager;
    public Department (Person manager) {
        _manager = manager;
    }
    public Person getManager() {
        return _manager;
    }
}
```

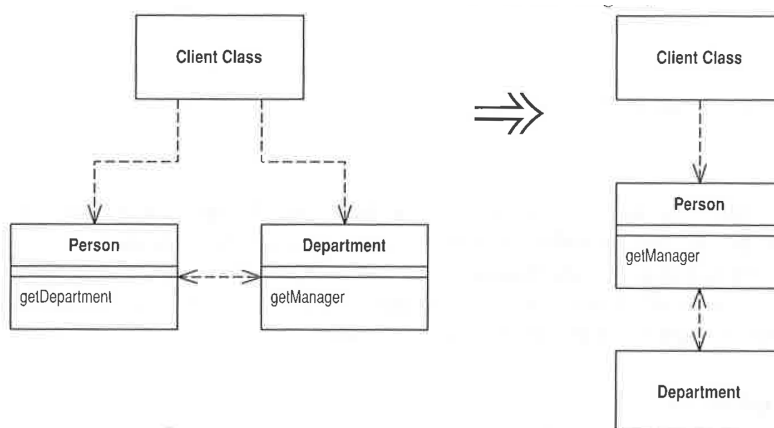
Cambiamenti a Department si ripercuotono sui client di Person



```
manager = john.getDepartment().getManager();
```

11

...e dopo



```
class Person { ...
    Department _department;
    public Person getManager() {
        return _department.getManager();
    }
}
```

Creo un metodo delegante getManager

1

```
manager = john.getDepartment().getManager();
```

Sistemo i client affinché chiamino getManager

2

```
class Person {
    Department _department;
    public Department getDepartment() {
        return _department;
}
```

Rimuovo da Person il metodo che espone getDepartment

3

4

Introduce Foreign Method

Situazione

Una classe Server ha bisogno di un metodo aggiuntivo, ma non è possibile modificare la classe

Soluzione

Crea un metodo nella classe Client con un'istanza della classe Server come primo parametro

Motivazione

Se il metodo mancante serve in più punti della classe Client, è necessario ripetere più volte il codice mancante nel Client

Pattern correlati

Introduce Local Extension (se manca più di un metodo)

13

Come operare il refactoring?

1. Crea un metodo nella classe Client che fa quanto occorre
2. Rendi il primo parametro del metodo una istanza della classe Server
3. Commenta il metodo come “foreign method; should be in server”

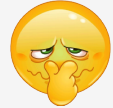
14

Esempio di Introduce Foreign Method

Prima e dopo

```
class Client { ...  
    Date newStart = new Date (previousEnd.getYear(), previousEnd.getMonth(),  
        previousEnd.getDay() + 1);
```

Questo pezzo di codice è ripetuto
più volte nella classe Client



```
class Client { ...  
    Date newStart = new Date (previousEnd.getYear(), previousEnd.getMonth(),  
        previousEnd.getDate() + 1);  
    Date newStart = nextDay(previousEnd);
```

Estraggo l'istruzione
in un metodo con
parametro Date
commentandolo
opportunamente

1,2,3

```
private static Date nextDay(Date arg) {  
    // foreign method, should be on Date  
    return new Date (arg.getYear(), arg.getMonth(), arg.getDay() + 1);  
}
```

Tipi di refactoring



- Per sistemare metodi troppo lunghi
- Per sistemare le responsabilità mal attribuite
- **Per riorganizzare i dati**
- Per semplificare statement condizionali
- Per rendere le chiamate ai metodi più semplici
- Per sistemare le gerarchie

Replace Data Value with Object

Situazione

Un dato va arricchito con altri dati o con comportamenti

Soluzione

Sostituisci il dato con un oggetto

Motivazione

Durante lo sviluppo ci si accorge che un semplice dato non è così semplice



17

Come operare il refactoring?

1. Crea una classe per il valore
2. Aggiungi un attributo final dello stesso tipo alla nuova classe
3. Aggiungi un metodo getter e un costruttore che ha come parametro il valore stesso;
4. Nella classe sorgente cambia il tipo dell'attributo col tipo della nuova classe;
5. Nella classe sorgente usa il metodo getter della classe creata;
6. Se il costruttore della classe sorgente inizializza l'attributo, inizializza l'attributo usando il costruttore della nuova classe;
7. Cambia il metodo setter per usare la nuova classe
8. Compila e testa;

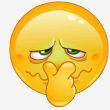
18

Esempio di Replace Data Value with Object

Prima...

```
class Order { ...  
    public Order (String customer) {  
        _customer = customer;  
    }  
    public String getCustomer() {  
        return _customer;  
    }  
    public void setCustomer(String arg) {  
        _customer = arg;  
    }  
    private String _customer;  
}
```

Ci accorgiamo che il cliente
non dovrebbe essere
semplicemente una stringa



```
private static int numberOfOrdersFor(Collection orders, String customer) {  
    int result = 0;  
    Iterator iter = orders.iterator();  
    while (iter.hasNext()) {  
        Order each = (Order) iter.next();  
        if (each.getCustomerName().equals(customer)) result++;  
    }  
    return result;  
}
```

Questo è un possibile client
di una classe che modella il
concetto di cliente

19

...e dopo

```
class Customer {  
    public Customer (String name) {  
        _name = name;  
    }  
    public String getName() {  
        return _name;  
    }  
    private final String _name;  
}
```

Creo una classe Customer con
un attributo `_name`, un
costruttore e un metodo getter

1,2,3

```
class Order { ...  
    private Customer _customer;  
  
    public String getCustomer() getCustomerName() {  
        return _customer _customer.getName();  
    }  
    public Order (String customer customerName) {  
        _customer = new Customer(customer customerName);  
    }  
    public void setCustomer (String arg customerName) {  
        _customer = new Customer(customer customerName);  
    }  
}
```

Cambio il tipo dell'attributo

4

Delego al nuovo getter

5

Uso il costruttore
di Customer

6

Cambio il setter per
usare Customer

7

8

Tipi di refactoring



- Per sistemare metodi troppo lunghi
- Per sistemare le responsabilità mal attribuite
- Per riorganizzare i dati
- **Per semplificare statement condizionali**
- Per rendere le chiamate ai metodi più semplici
- Per sistemare le gerarchie

21

Introduce Null Object

Situazione

Il codice contiene ripetuti controlli riguardanti il null value

Soluzione

Sostituisci il null value con un null object

Motivazione

In molti posti del codice si controlla per la presenza di un oggetto prima di mandare un messaggio all'oggetto

Se l'oggetto non è presente si reagisce quasi sempre allo stesso modo

22

Come operare il refactoring?

1. Definisci una sottoclasse della classe sorgente che rappresenti la versione null della classe
2. Definisci una operazione isNull sulla classe sorgente che ritorna false e una operazione isNull sulla sottoclasse che ritorna true
3. Compila e testa;
4. Aggiungi un factory method per il null object;
5. Modifica i punti che prevedono la restituzione di una istanza della classe sorgente per ritornare il null object invece del null value;
6. Modifica i punti che confrontano una istanza della classe sorgente con il null value per invocare l'operazione isNull
7. Compila e testa;
8. Per tutti i punti in cui i Client fanno qualcosa se not null e qualcosa in alternativa se null, rimuovi lo statement condizionale grazie al polimorfismo
9. Compila e testa;

23

Esempio di Null Object

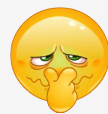
Prima e per iniziare...

```
class Site { ...
    Customer getCustomer() {
        return _customer;
    }
    Customer _customer;

class Customer { ...
    public String getName() {...}
    public BillingPlan getPlan() {...}

Customer customer = site.getCustomer();
...
String customerName;
if (customer == null) customerName = "occupant";
else customerName = customer.getName();
if (customer != null) customer.setPlan(BillingPlan.special());
```

Parecchi client controllano per la presenza del customer prima di inviare messaggi getName e setPlan



```
class NullCustomer extends Customer {
    public boolean isNull() {
        return true;
    }
}

class Customer { ...
    public boolean isNull() {
        return false;
    }
    protected Customer() {} //needed by the NullCustomer
```

Definisco una sottoclasse per il null customer

1

Definisco un'operazione isNull su Customer e NullCustomer

2

3

...durante e dopo

```
class Customer { ...  
    static Customer newNull() {  
        return new NullCustomer();  
    }  
}
```

Factory method: i client non debbono conoscere la classe NullCustomer

4

```
class Site { ...  
    Customer getCustomer() {  
        return _customer;  
        return (_customer == null) ?  
            Customer.newNull(): _customer;  
    }  
}
```

Modifico tutti i punti in cui si chiede un Customer per ritornare il null Customer piuttosto che il null value

5

```
Customer customer = site.getCustomer();  
BillingPlan plan;  
String customerName;  
if (customer == null customer.isNull()) customerName = "occupant";  
else customerName = customer.getName();  
...  
if (customer == null customer.isNull()) customer.setPlan(BillingPlan.special());
```

Modifico la condizione customer == null invocando isNull

6

```
class NullCustomer { ...  
    public String getName(){  
        return "occupant";  
    }  
    public void setPlan (BillingPlan arg) {}  
    String customerName = customer.getName();  
    customer.setPlan(BillingPlan.special());  
}
```

Aggiungo i metodi opportuni alla classe NullCustomer e rimuovo i costrutti condizionali

7

8

9

Introduce Assertion

Situazione

Del codice fa assunzioni riguardo lo stato del programma

Soluzione

Rendi le assunzioni esplicite tramite un'asserzione

Esempio

```
double getExpenseLimit() {  
    // should have either expense limit or a primary project  
    return (_expenseLimit != NULL_EXPENSE) ? _expenseLimit: _primaryProject.getMemberExpenseLimit();  
}
```

Prima

```
double getExpenseLimit() {  
    Assert.isTrue (_expenseLimit != NULL_EXPENSE || _primaryProject != null);  
    return (_expenseLimit != NULL_EXPENSE) ? _expenseLimit: _primaryProject.getMemberExpenseLimit();  
}
```

Dopo

Tipi di refactoring



- Per sistemare metodi troppo lunghi
- Per sistemare le responsabilità mal attribuite
- Per riorganizzare i dati
- Per semplificare statement condizionali
- **Per rendere le chiamate ai metodi più semplici**
- Per sistemare le gerarchie

27

Separate Query from Modifier

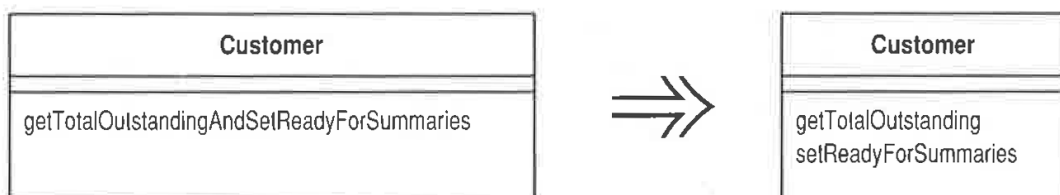
Situazione

Un metodo ritorna un valore e cambia lo stato di un oggetto

Soluzione

Crea due metodi, uno per la query e uno per la modifica

Esempio



28

Replace Parameter with Explicit Method

Situazione

Un metodo esegue codice differente in base al valore di un parametro

Soluzione

Crea un metodo separato per ciascun valore del parametro

Esempio

```
void setValue (String name, int value) {  
    if (name.equals("height"))  
        _height = value;  
    if (name.equals("width"))  
        _width = value;  
    Assert.shouldNeverReachHere();  
}
```

Prima

```
void setHeight(int arg) {  
    _height = arg;  
}  
void setWidth (int arg) {  
    _width = arg;  
}
```

Dopo

29

Preserve Whole Object

Situazione

Si richiede più valori a un oggetto per poi invocare un metodo con questi valori come parametri

Soluzione

Invoca il metodo con l'intero oggetto piuttosto

Esempio

```
int low = daysTempRange().getLow();  
int high = daysTempRange().getHigh();  
withinPlan = plan.withinRange(low, high);
```

Prima

```
withinPlan = plan.withinRange(daysTempRange());
```

Dopo

30

Introduce Parameter Object

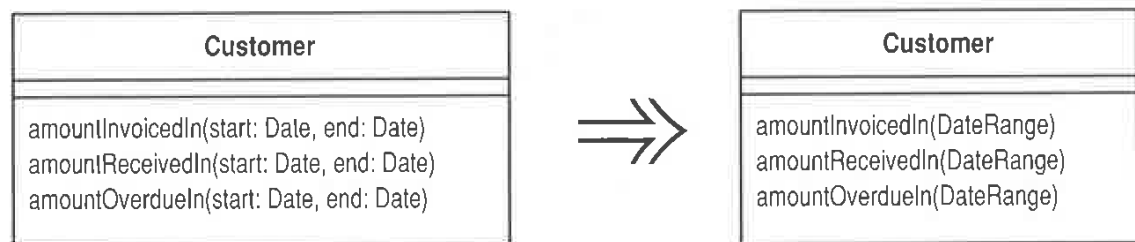
Situazione

Un gruppo di parametri tende a essere passato assieme

Soluzione

Raggrupparli in una classe e usa l'oggetto nella invocazione

Esempio



31

Tipi di refactoring



- Per sistemare metodi troppo lunghi
- Per sistemare le responsabilità mal attribuite
- Per riorganizzare i dati
- Per semplificare statement condizionali
- Per rendere le chiamate ai metodi più semplici
- **Per sistemare le gerarchie**

32

Pull Up Method

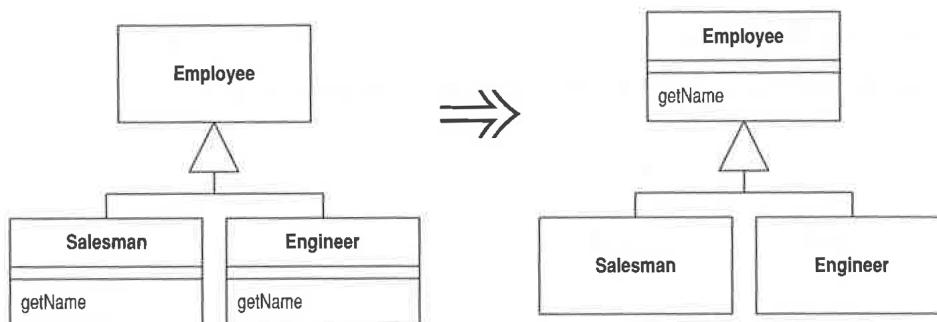
Situazione

Due metodi nelle sottoclassi producono gli stessi risultati

Soluzione

Sposta i metodi nella superclasse

Esempio



33

Pull Up Constructor Body

Situazione

I costruttori nelle sottoclassi hanno per lo più lo stesso corpo

Soluzione

Crea un costruttore per la superclasse e invoca questo dai costruttori delle sottoclassi

Esempio

```
class Manager extends Employee { ...
    public Manager (String name, String id, int grade) {
        _name = name;
        _id = id;
        _grade = grade;
    }
}
```

Prima

```
public Manager (String name, String id, int grade) {
    super (name, id);
    _grade = grade;
}
```

Dopo

Extract Subclass

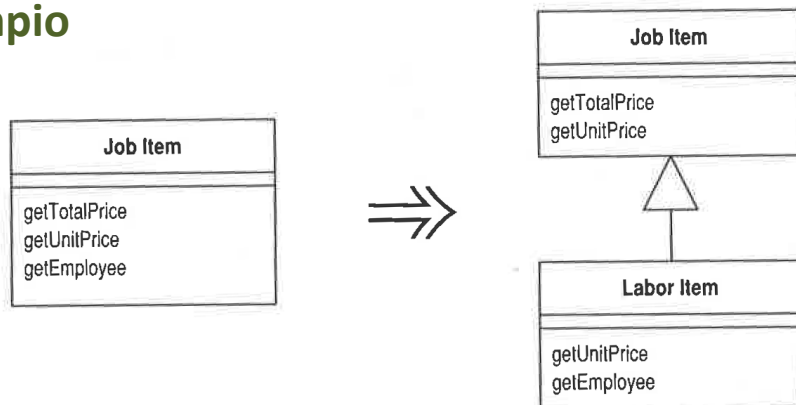
Situazione

Una classe ha features che sono usate solo da alcune istanze

Soluzione

Crea una sottoclasse per quell'insieme di features

Esempio



35

Extract Superclass

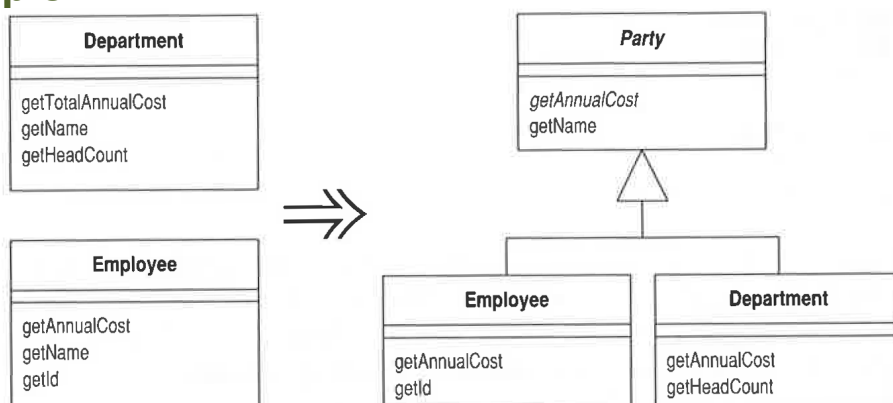
Situazione

Due classi hanno features simili

Soluzione

Crea una superclasse e sposta le feature comuni nella superclasse

Esempio



36

Extract Interface

Situazione

Più client usano una stessa porzione di interfaccia di una classe o due client hanno parti di loro interfacce in comune

Soluzione

Estrai quelle porzioni di interfaccia in una interface

Esempio

